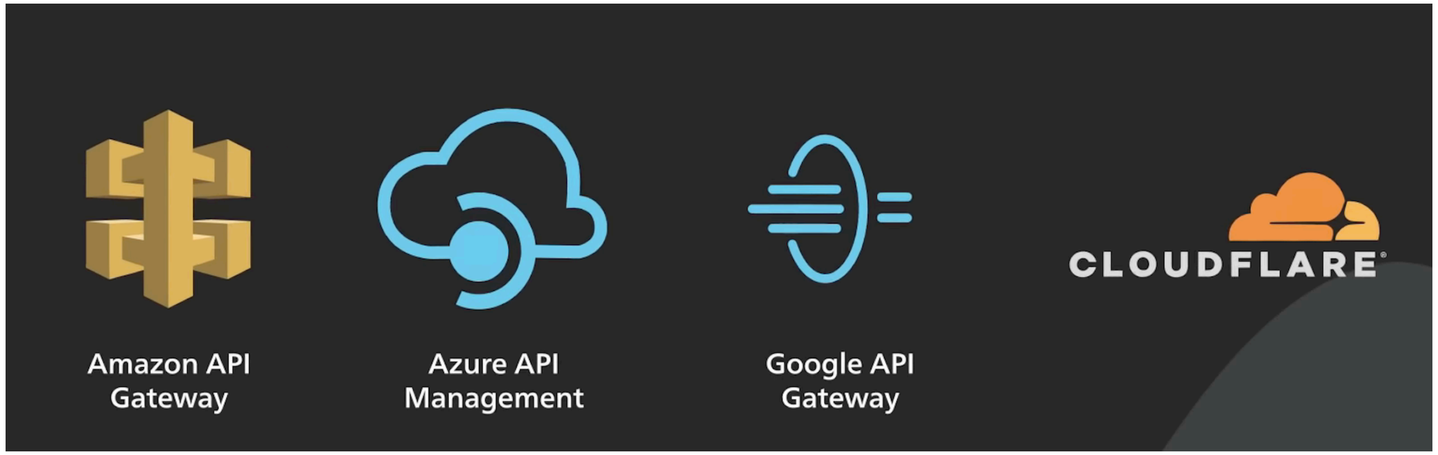
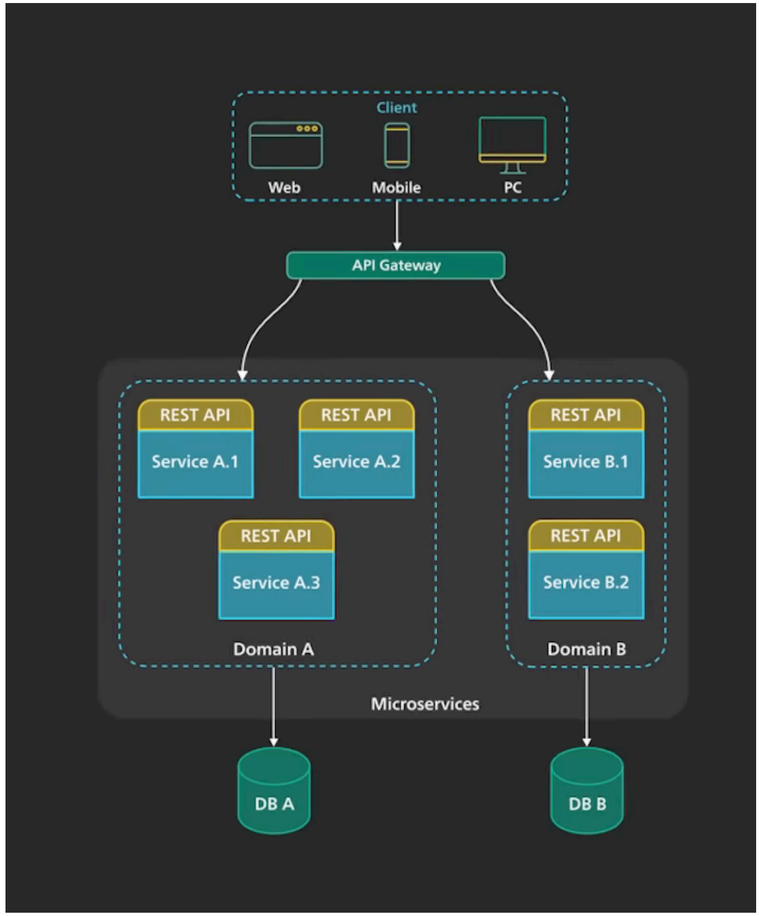
APII Gateway

Api Gateway


Amazon API Gateway
Azure API Management
Google API Gateway
CLOUDFLARE

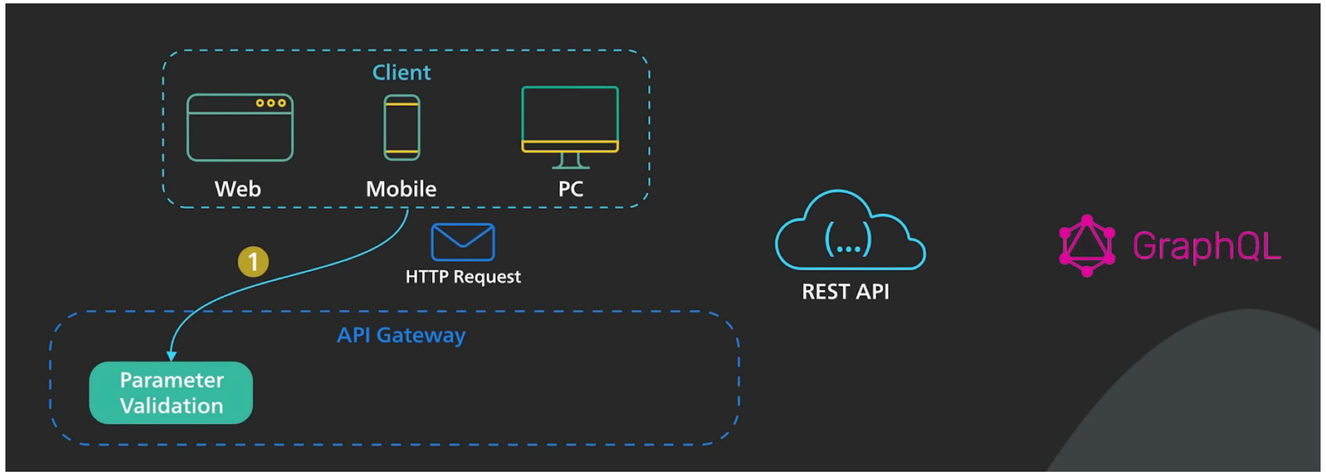
Api Gateway

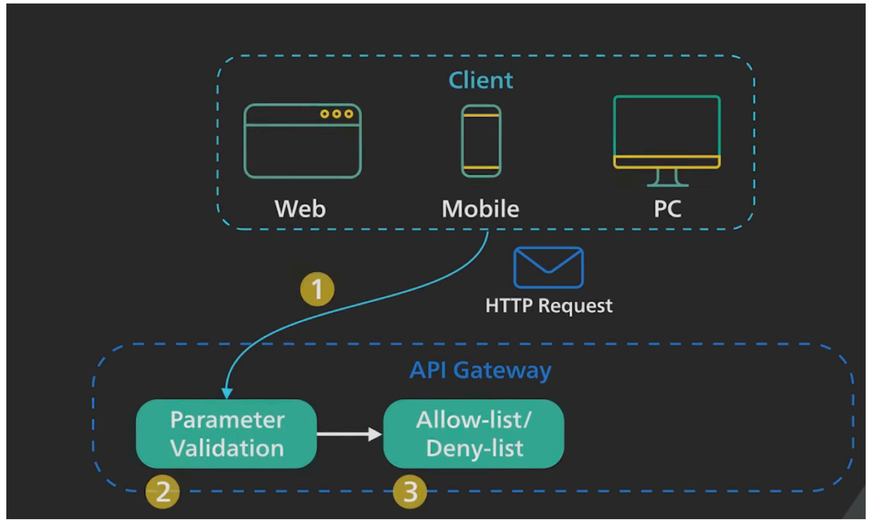


API Gateway - osnovne funkcionalnosti

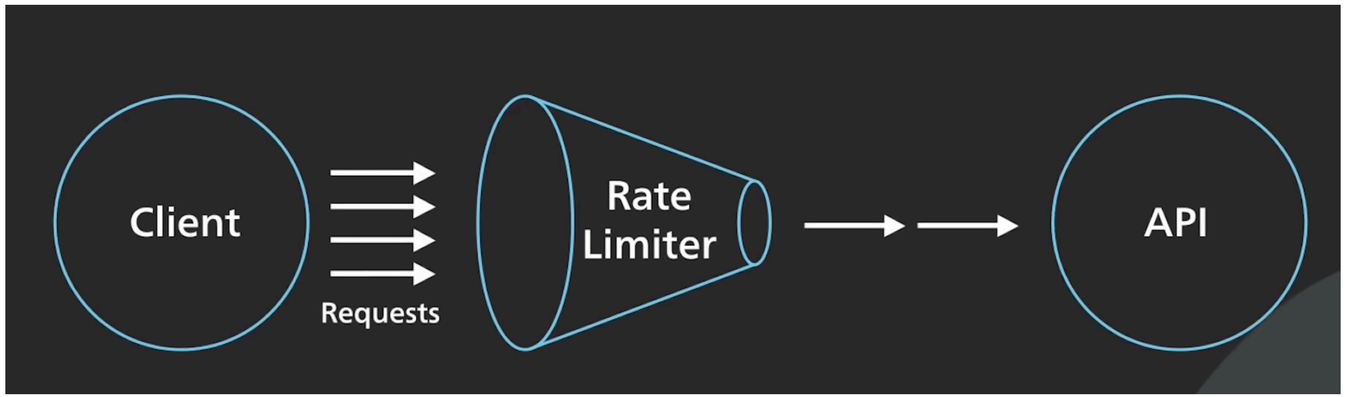
- Avtentikacija in varnost
- Load balancing
- Protocol translation + service discovery
- Nadzor, log, analitika, billing
- Caching

Api Gateway

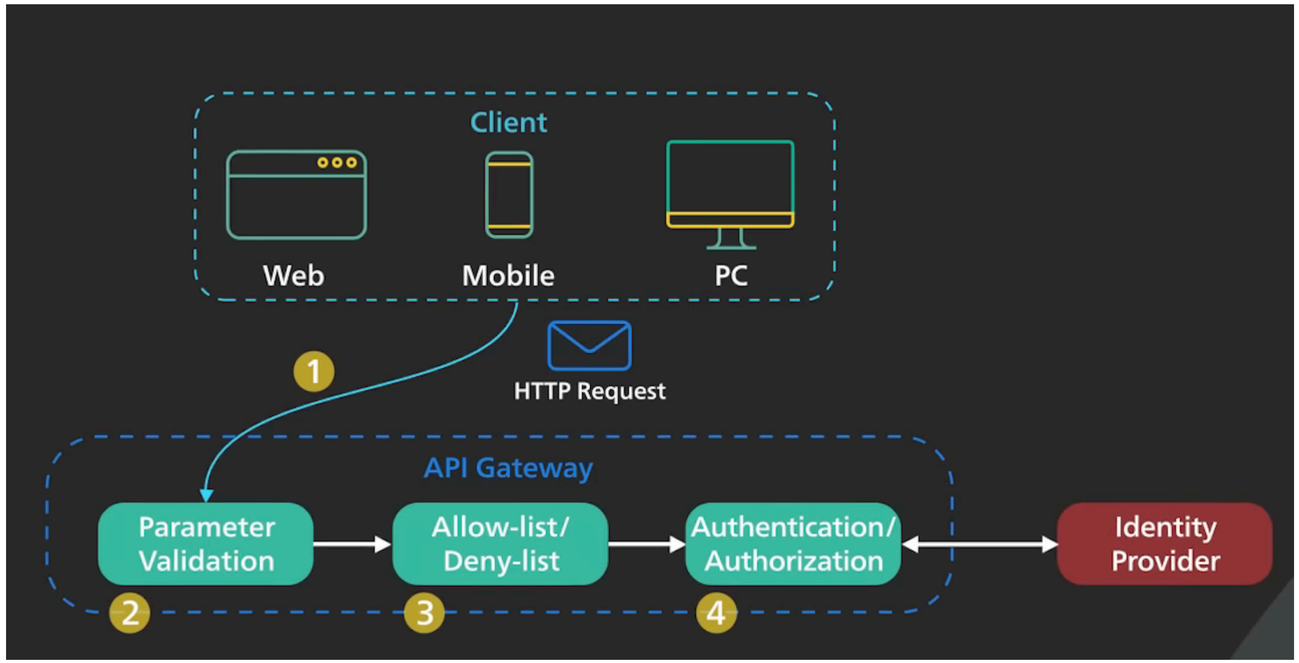

Client
Web
Mobile
PC
1
HTTP Request
API Gateway
Parameter Validation
REST API
GraphQL


Client
Web
Mobile
PC
1
HTTP Request
API Gateway
Parameter Validation
Allow-list/ Deny-list
2
3

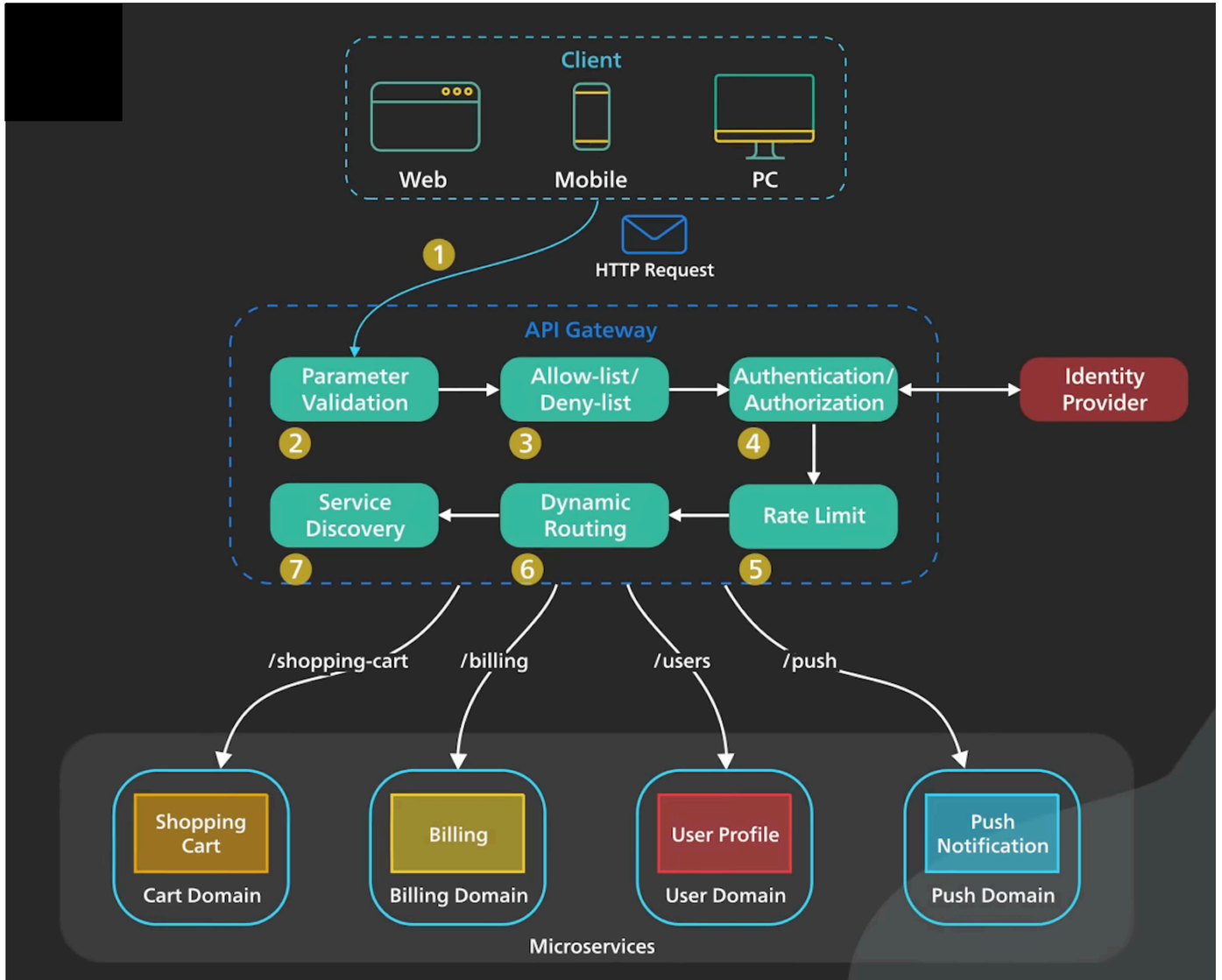
Api Gateway



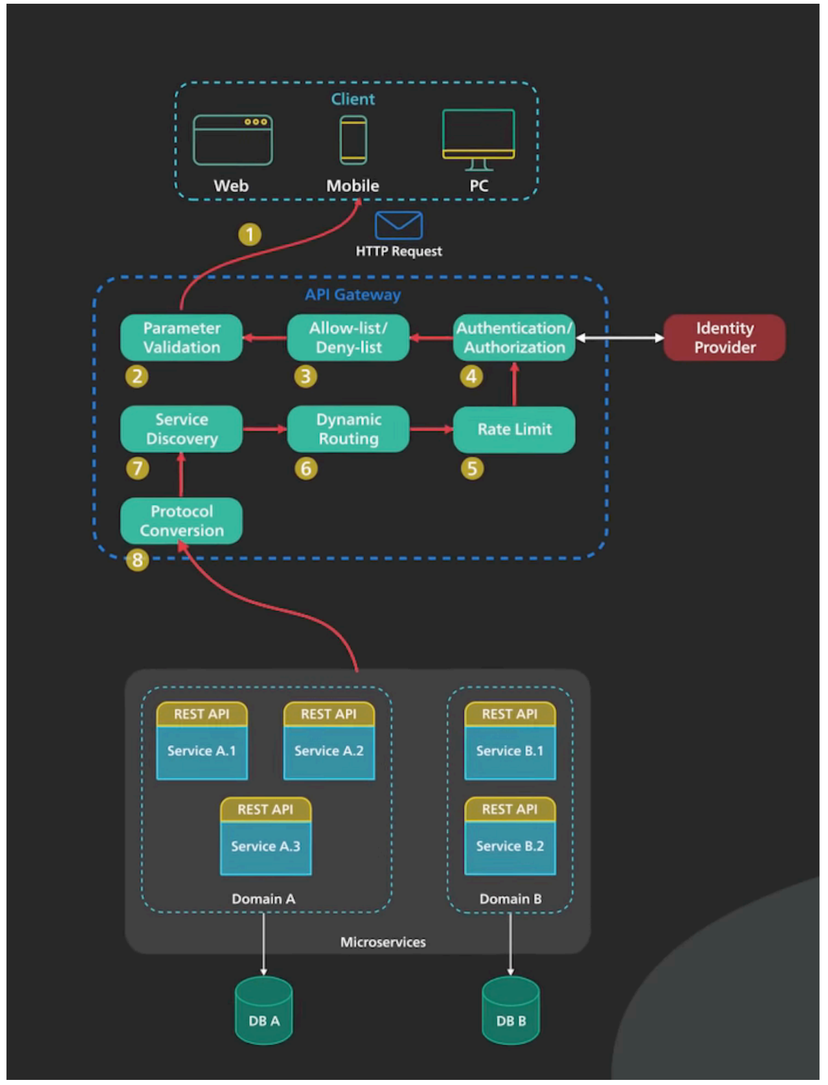
Api Gateway


Client
Web
Mobile
PC
HTTP Request
1
API Gateway
Parameter Validation
Allow-list/ Deny-list
Authentication/ Authorization
Identity Provider
2
3
4

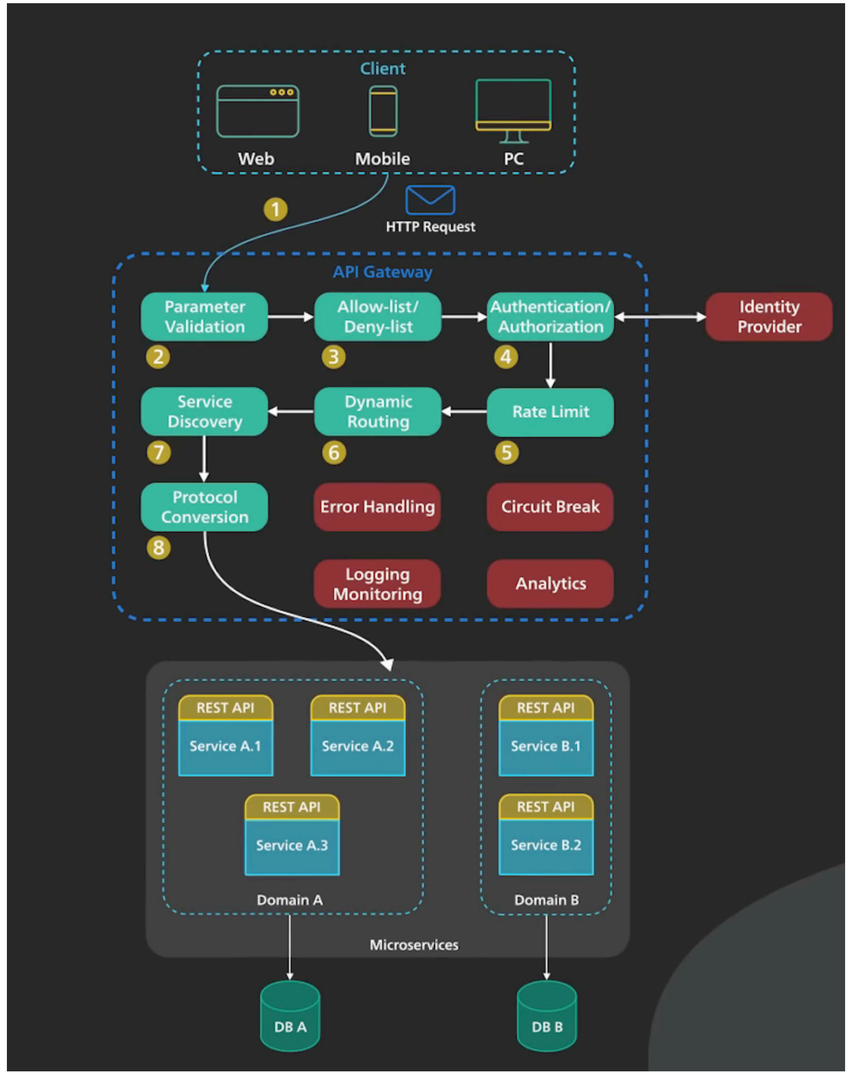
Api Gateway


Client
Web
Mobile
PC
1
HTTP Request
API Gateway
Parameter Validation
2
Allow-list/ Deny-list
3
Authentication/ Authorization
4
Identity Provider
Service Discovery
7
Dynamic Routing
6
Rate Limit
5
/shopping-cart
/billing
/users
/push
Shopping Cart
Cart Domain
Billing
Billing Domain
User Profile
User Domain
Push Notification
Push Domain
Microservices

Api Gateway

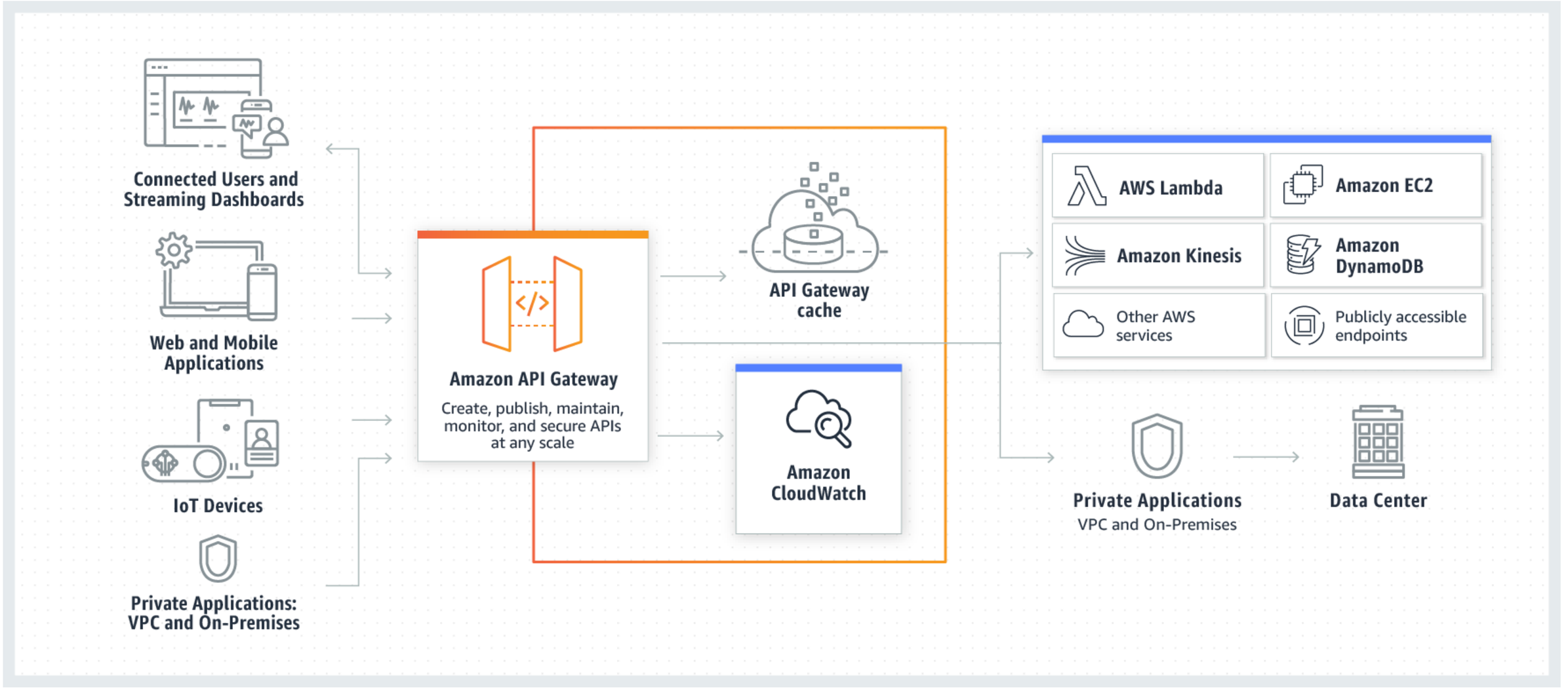

Client
Web
Mobile
PC
1
HTTP Request
API Gateway
Parameter Validation
2
Allow-list/ Deny-list
3
Authentication/ Authorization
4
Identity Provider
Service Discovery
7
Dynamic Routing
6
Rate Limit
5
Protocol Conversion
8
REST API
Service A.1
REST API
Service A.2
REST API
Service A.3
Domain A
REST API
Service B.1
REST API
Service B.2
Domain B
Microservices
DB A
DB B

Api Gateway


Client
Web
Mobile
PC
1
HTTP Request
API Gateway
Parameter Validation
2
Allow-list/ Deny-list
3
Authentication/ Authorization
4
Identity Provider
Service Discovery
7
Dynamic Routing
6
Rate Limit
5
Protocol Conversion
8
Error Handling
Circuit Break
Logging Monitoring
Analytics
REST API
Service A.1
REST API
Service A.2
REST API
Service A.3
Domain A
REST API
Service B.1
REST API
Service B.2
Domain B
Microservices
DB A
DB B

Api Gateway

AWS



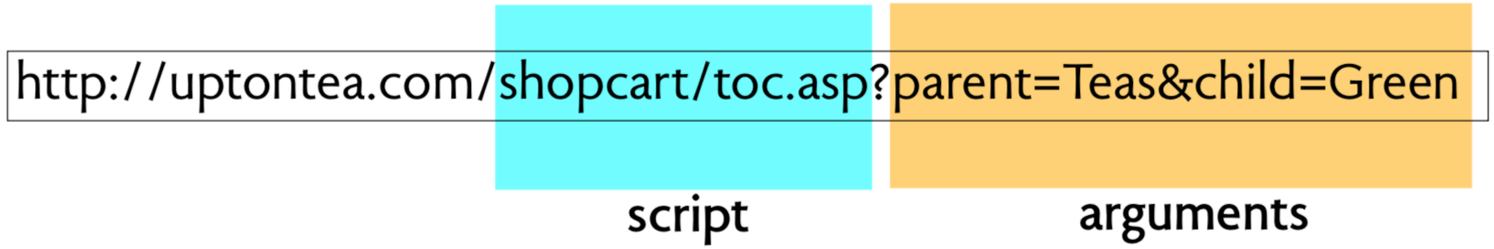
REST

Old school - 1990s

Kaj je bilo možno

- Samo GET
- Mosaic - FORM
- GET
  - Query strung
- POST - pošiljanje podatkov

Kako je bilo



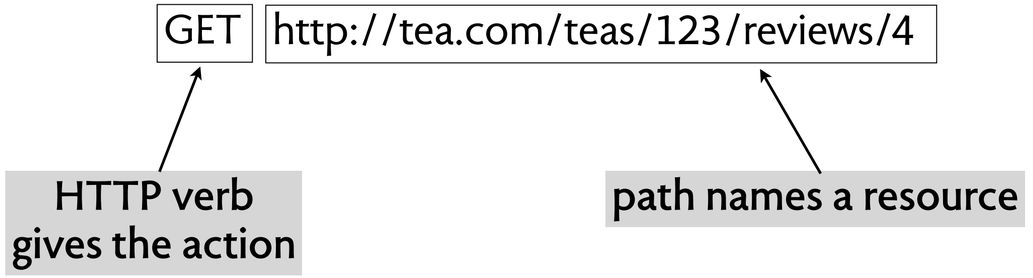
- Novi ukazi
  - DELETE pages/3
  - POST /deletePage
  - GET /deletePage?id=3

- GET /foo.asp?action=delete&page=3

New age

CRUD	*SQL*	*HTTP*
create	insert	POST
read	select	GET
update	update	PUT
delete	delete	DELETE

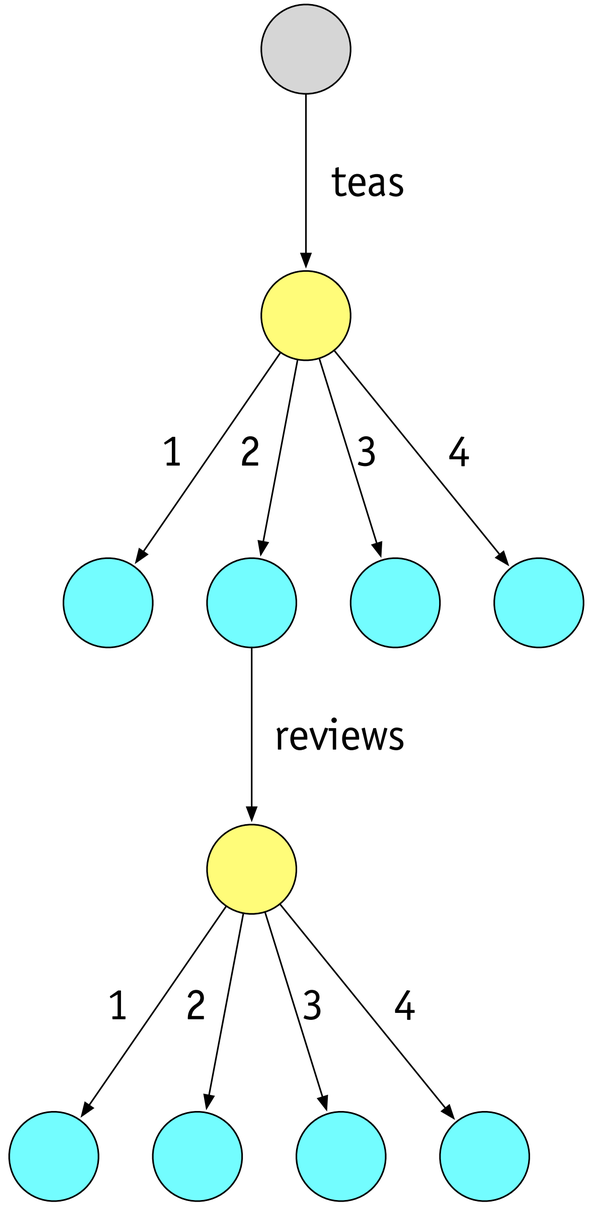
REST



- Poti

- › http://tea.com/teas
  › http://tea.com/teas/123/reviews
  › http://tea.com/teas/green

- › http://tea.com/teas/123
  › http://tea.com/teas/123/reviews/4

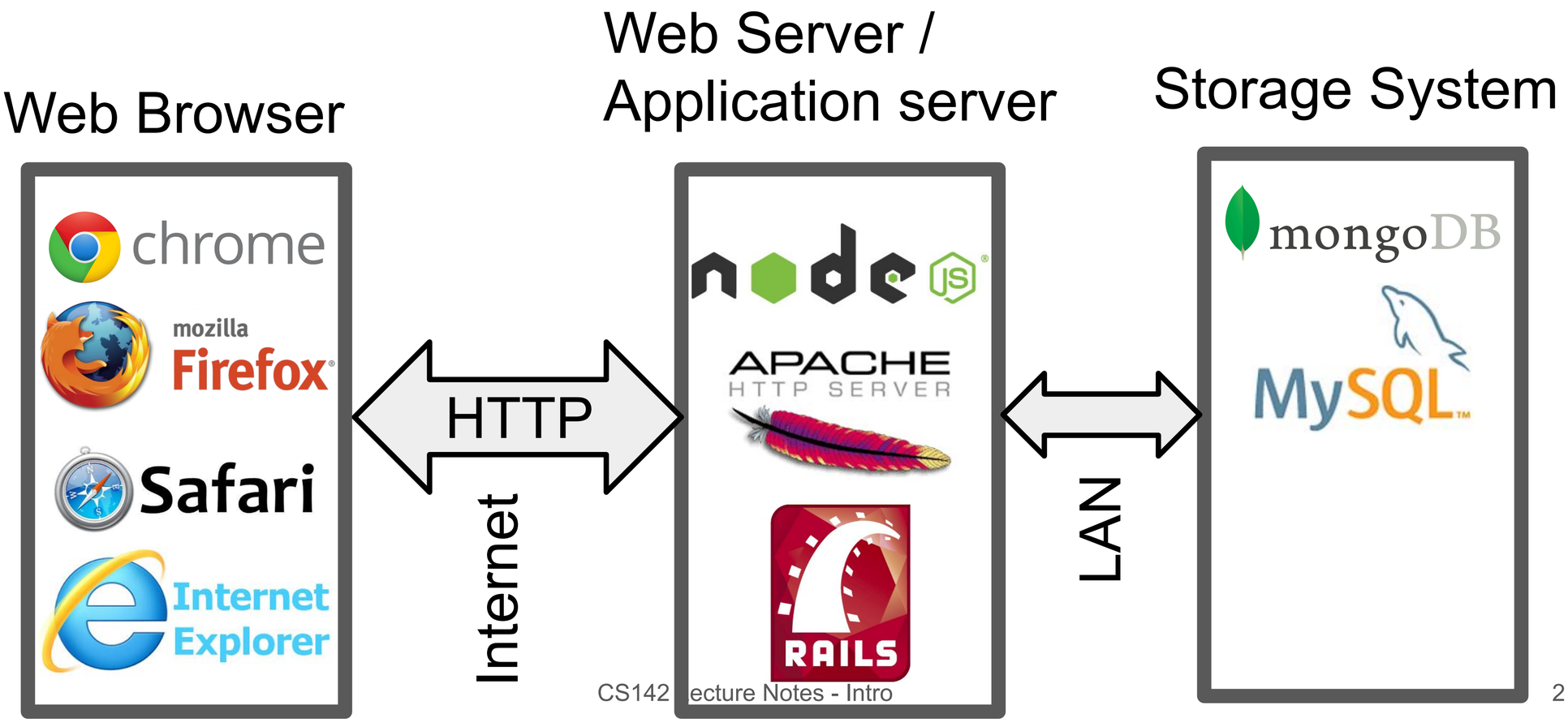


Ukaze

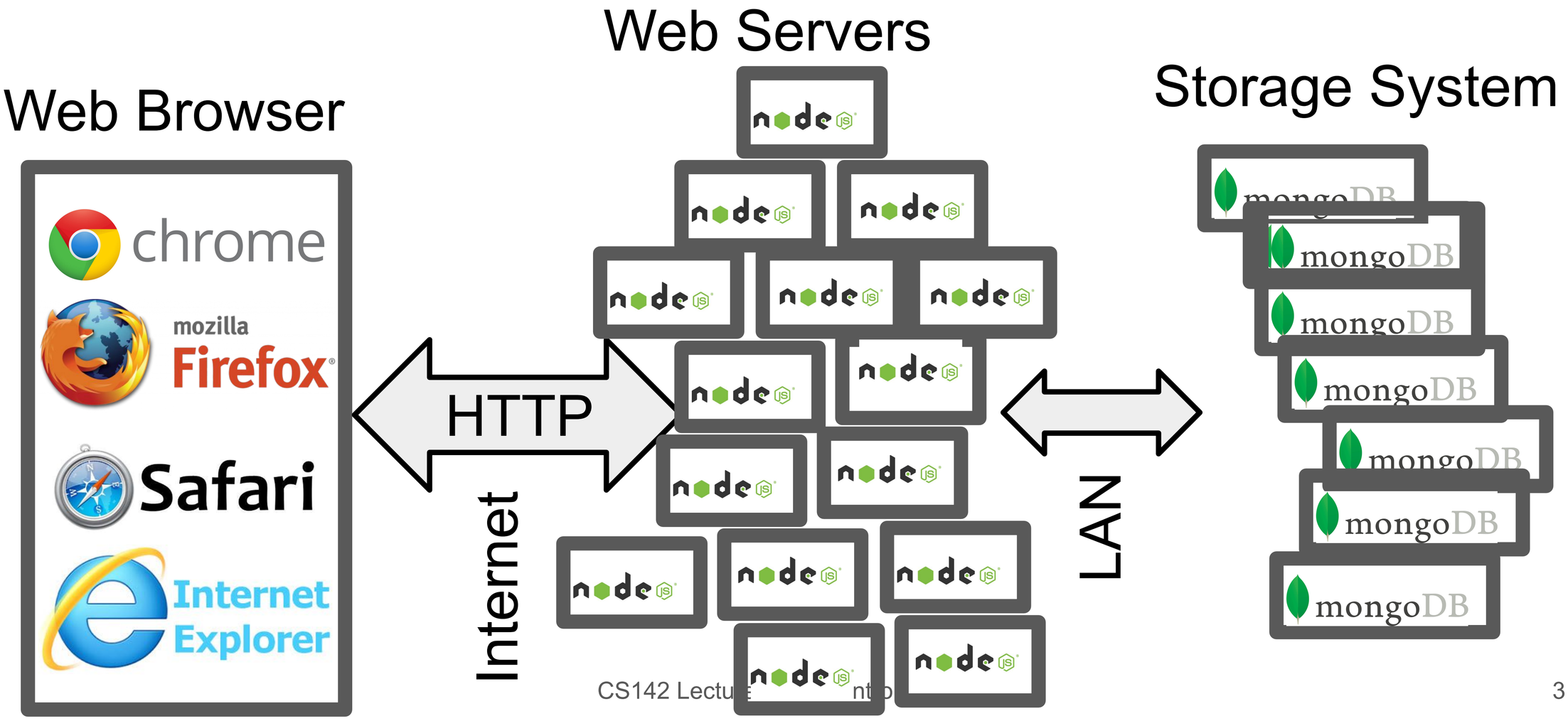
Idempotent - stateless

- Prikaži review tea #123
  - GET http://tea.com/teas/123/reviews
- Vpiši nov review tea #123
  - POST http://tea.com/teas/123/reviews
- Spremeni review #4 of tea #123
  - PUT http://tea.com/teas/123/reviews/4
- Izbriši review #4 of tea #123
  - DELETE http://tea.com/teas/123/reviews/4

Tipična arhitektura



Large scale


Web Browser
Web Servers
Storage System
chrome
mozilla
Firefox
Safari
Internet
Explorer
HTTP
Internet
LAN
mongoDB
mongoDB
mongoDB
mongoDB
mongoDB
mongoDB

Web service

Težave

- REST Endpoint Consensus
  - /user/123
  - /user/id/123
  - /user/?id=123
- REST API Verzije - /2.0/user/123
- REST API Varnost
- Večkratni ukazi in nepotrebne podatke

Web service

Varnost

- HTTPS
- Authentication method
- CORS
- Minimalna potrebna funkcionalnost
- Validacija vseh podatkov
- exposing API tokens in client-side JavaScript
    - https://kraji.eu/city_distance/slo
    - https://api.tomtom.com/routing/1/calculateRoute/%s,%s:%s,%s/json?key=V7IYZYMWd62wOSljkeLxSWOwMDe0U921&language=en-GB&traffic=false
- block access from unknown domains or IP addresses
- block large payloads
- rate-limiting
- respond with an appropriate HTTP status code and caching header
- log requests and investigate failures.

Open API 3.0

Opis APIs

- Describe RESTful HTTP APIs in a machine-readable way
- Then the machine can read it and write:
  - API reference documentation
  - Code SDKs
  - Postman collections
  - Mock servers

API Spec Languages

- **OpenAPI** is an open standard
- **RAML** from Mulesoft
- **API Blueprint** from Apiary

Struktura API

- JSON ali YAML datoteka, ki vsebuje vso specifikacijo (spodaj je YAML)

```yaml
openapi: 3.0.2
info:
  title: HECAT WP3 Apis
  version: 1.0.0
  description: API for HECAT WP3 usage
paths:
  /hecat/api/v1/default_dss_values/{variable_name}/:
    get:
      operationId: retrieveDefaultDSSValues
      description: ''
      parameters:
      - name: variable_name
...
Še več kot tisoč vrstic naprej
```

Endpoint

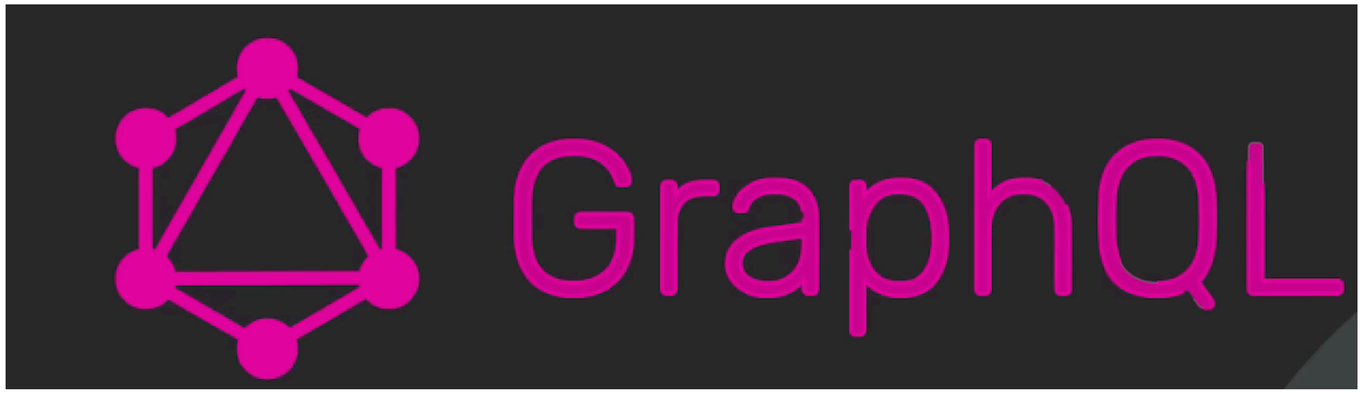
```
/hecat/api/v1/input_structure:
    get:
      operationId: listDSSStructures
      description: "Returns the JSON structure that is expected by the raw model.\n\
        \nThe \n\nThe user request should contain the same keys as the response here.\n\
        The applicable values are listed as in the response plus the user might add\
        \ ``*`` for cases where the input is either missing or its value is irrelevant."
      parameters: []
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/DSSIn'
          description: ''
      tags:
      - F03
```

Komponente

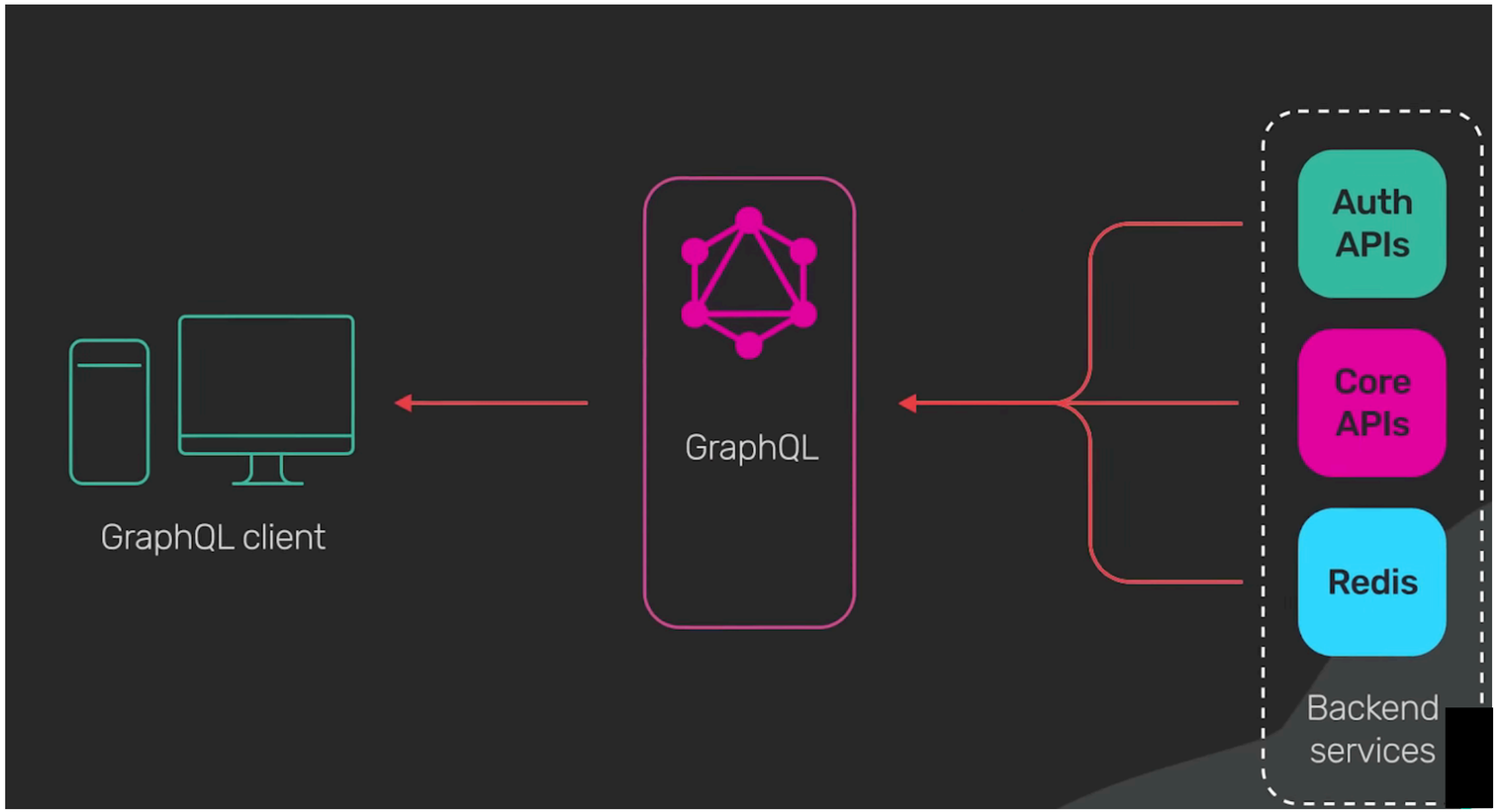
```yaml
components:
  schemas:
    DSSIn:
      type: object
      properties:
        skp_code:
          type: array
          items:
            type: string
          default:
          - '5120.01'
          - '9412.01'
        up_enota:
          type: string
          default: '64'
        wishes:
          type: array
          items:
            type: string
          default:
          - '5120.01'
          - '9412.01'
        wishes_location:
          type: array
          items:
            type: string
          default:
          - '0'
```

Orodja

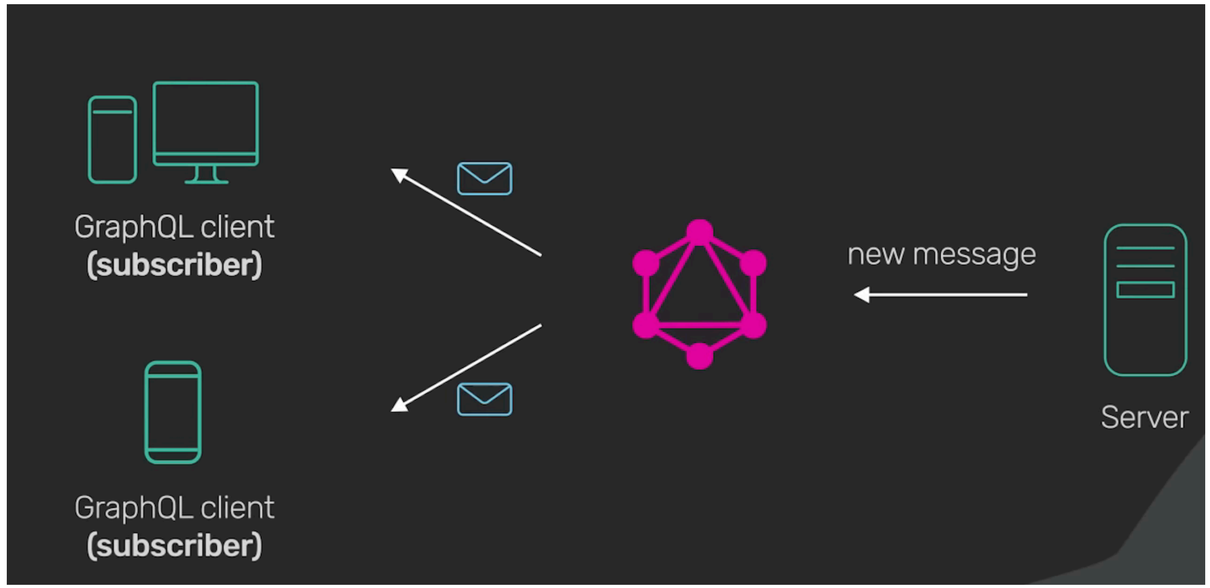
- Stoplight is highly recommended if you like a web interface https://stoplight.io
- Atom with linter-swagger https://atom.io
- SwaggerUI, SwaggerHub, etc https://swagger.io/tools/
- APICurio Studio gets good reviews https://www.apicur.io/


GraphQL

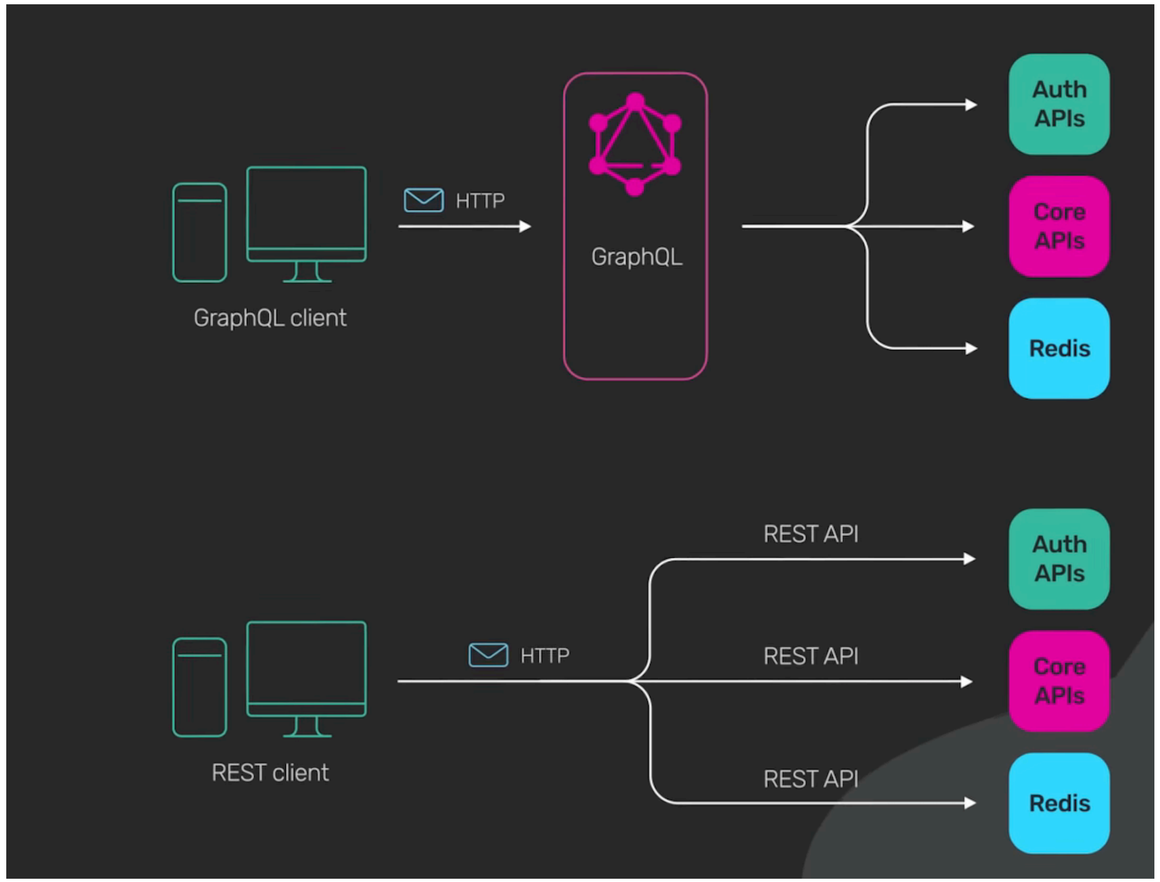
GraphQL


GraphQL client
GraphQL
Auth APIs
Core APIs
Redis
Backend services

GraphQL


GraphQL client
(subscriber)
GraphQL client
(subscriber)
new message
Server

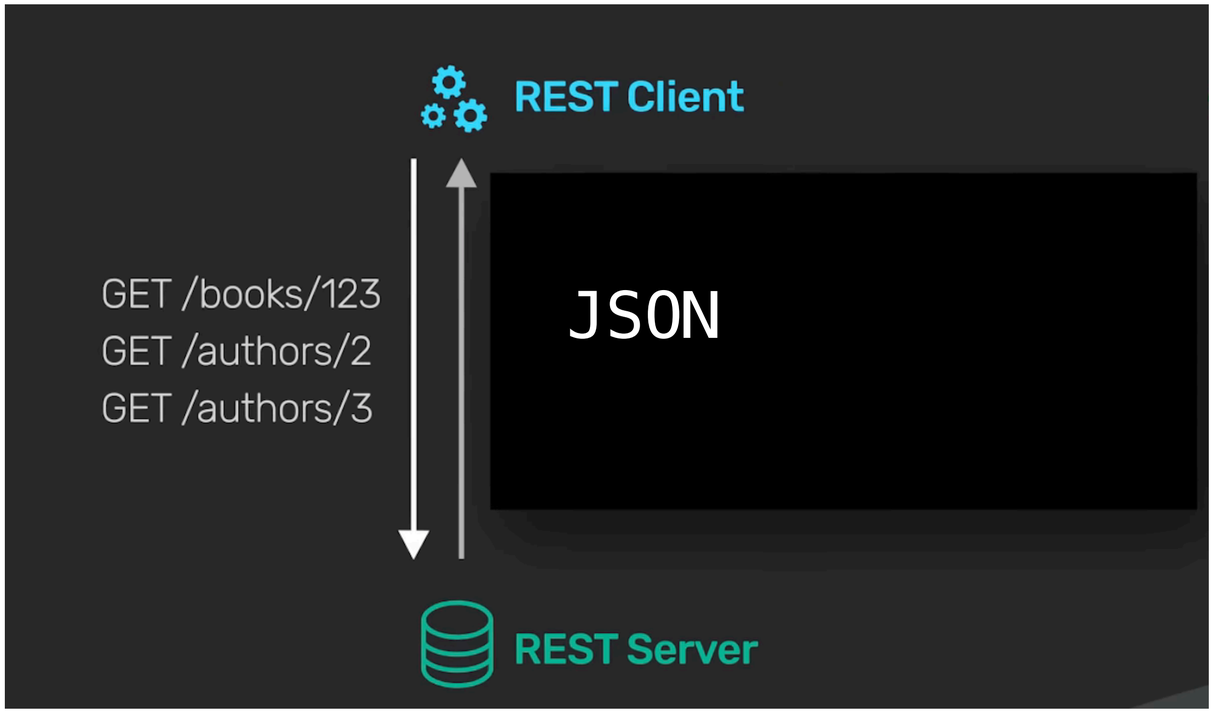
GraphQL


HTTP
GraphQL
GraphQL client
Auth APIs
Core APIs
Redis
REST API
REST API
REST API
HTTP
REST client
Auth APIs
Core APIs
Redis

GraphQL

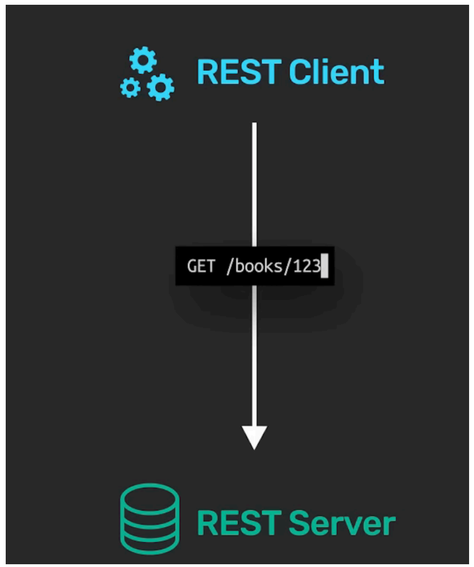

REST
API
https://example.com/api/v3/products
https://example.com/api/v3/users

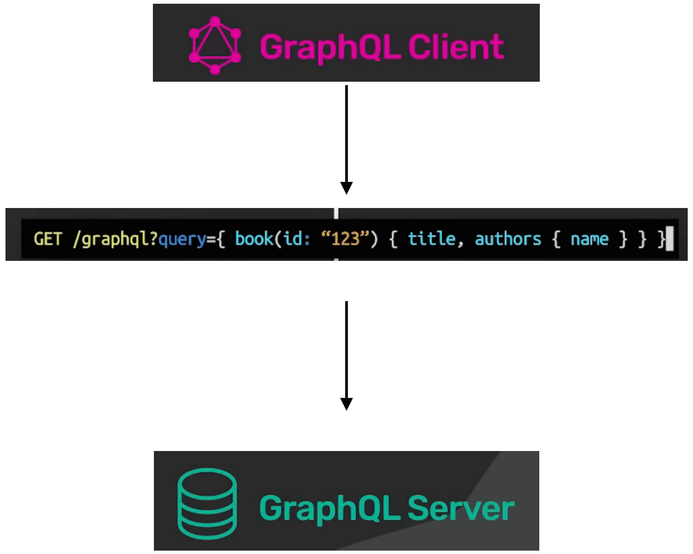
GraphQL


REST Client
GET /books/123
GET /authors/2
GET /authors/3
JSON
REST Server


GraphQL Client
type Book {
id: ID
title: String
authors: [Author]
}
type Author {
id: ID
name: String
books: [Book]
}

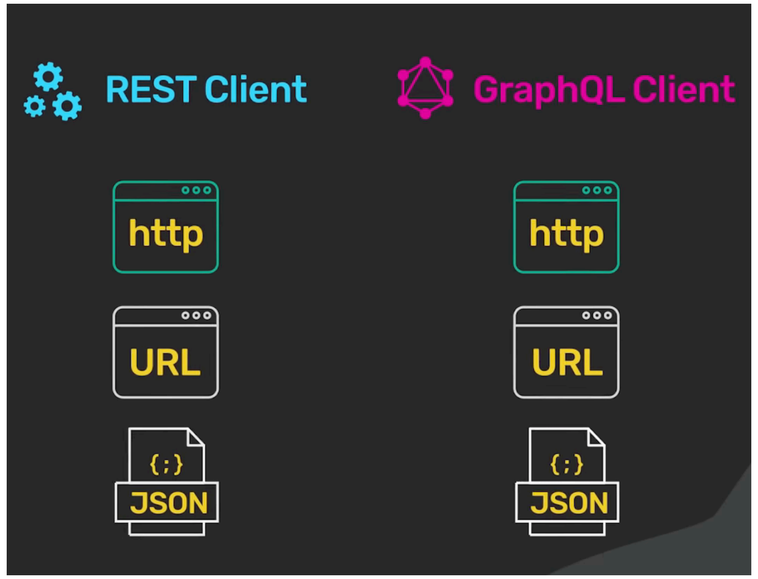
GraphQL


REST Client
GET /books/123
REST Server


GraphQL Client
GET /graphql?query={ book(id: “123”) { title, authors { name } } }
GraphQL Server

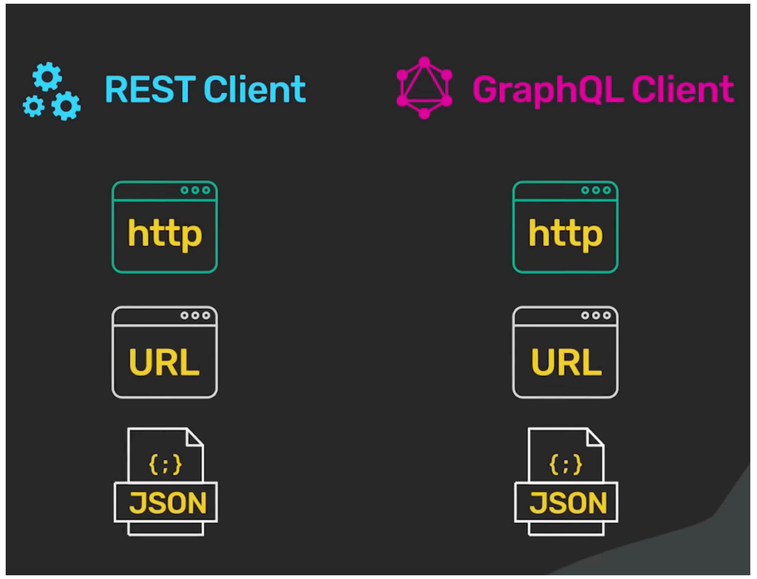
GraphQL

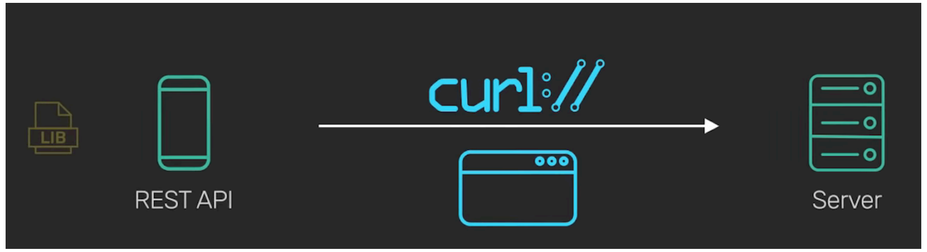
```
GET /graphql?query={ book(id: “123”) { title, authors { name } } }
```



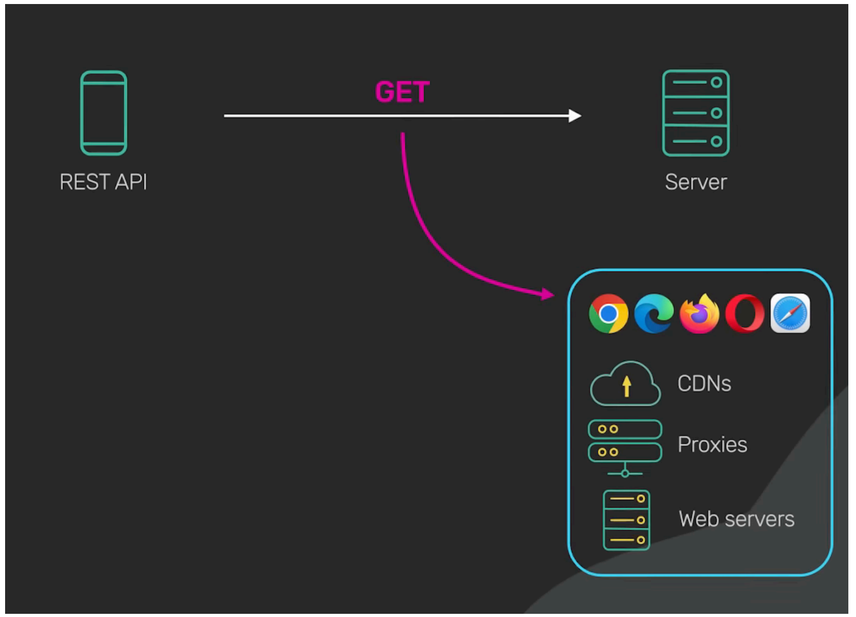
GraphQL

```
GET /graphql?query={ book(id: “123”) { title, authors { name } } }
```





Caching



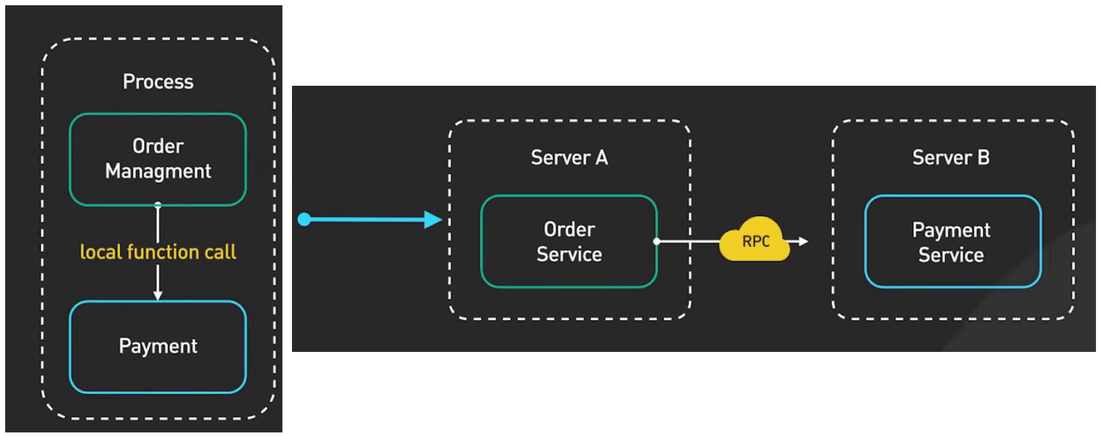
Google

2016

Google NETFLIX Square

CISCO Cockroach LABS

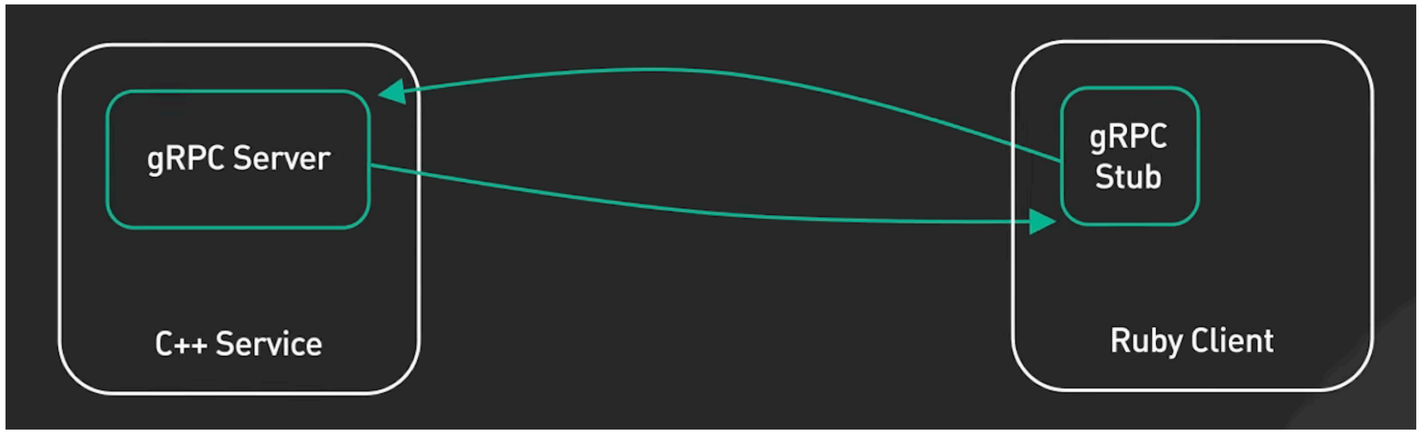
gRPC


Process
Order
Managment
local function call
Payment
Server A
Order
Service
RPC
Server B
Payment
Service

gRPC

- gRPC Core - C, C++, Ruby, Node.js, Python, PHP, C#, Objective-C
- gRPC Java - The Java gRPC implementation. HTTP/2 based RPC
- gRPC Kotlin - The Kotlin gRPC implementation. Based on gRPC Java
- gRPC Node.js - gRPC for Node.js
- gRPC Go - The Go language implementation of gRPC. HTTP/2 based RPC
- gRPC Swift - The Swift language implementaion of gRPC
- gRPC Dart - The Dart language implementaion of gRPC
- gRPC C# - The DaC$#rt language implementaion of gRPC
- gRPC Web - gRPC for Web Clients
- gRPC Ecosystem - gRPC for Ecosystem that complements gRPC
- gRPC contrib - Known useful contributions around github
- grpc_cli - gRPC CLI tool

gRPC




```
message Author {
  string name = 1;
  int32 id = 2;
}
```

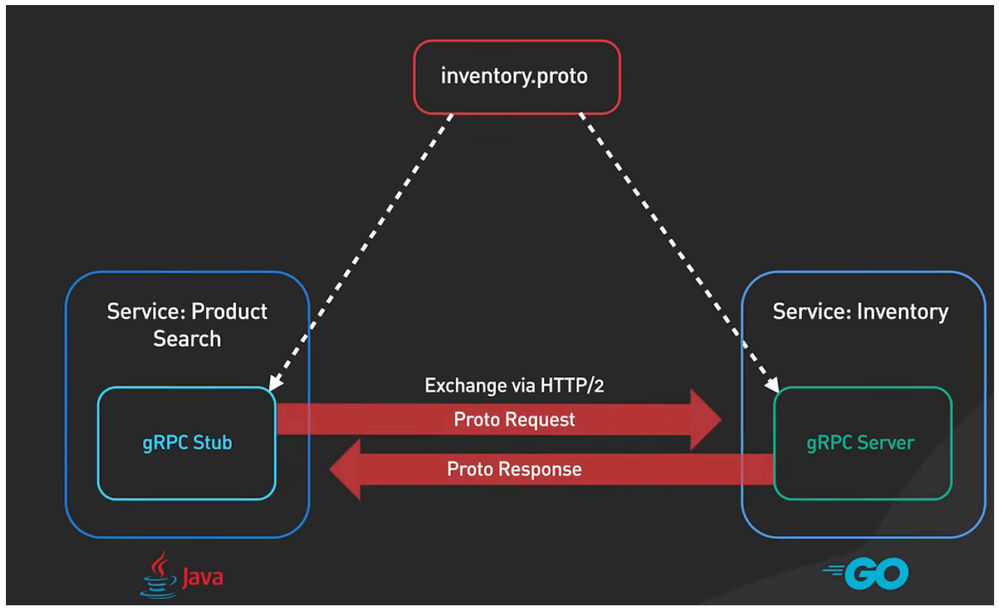
gRPC

publisher.proto

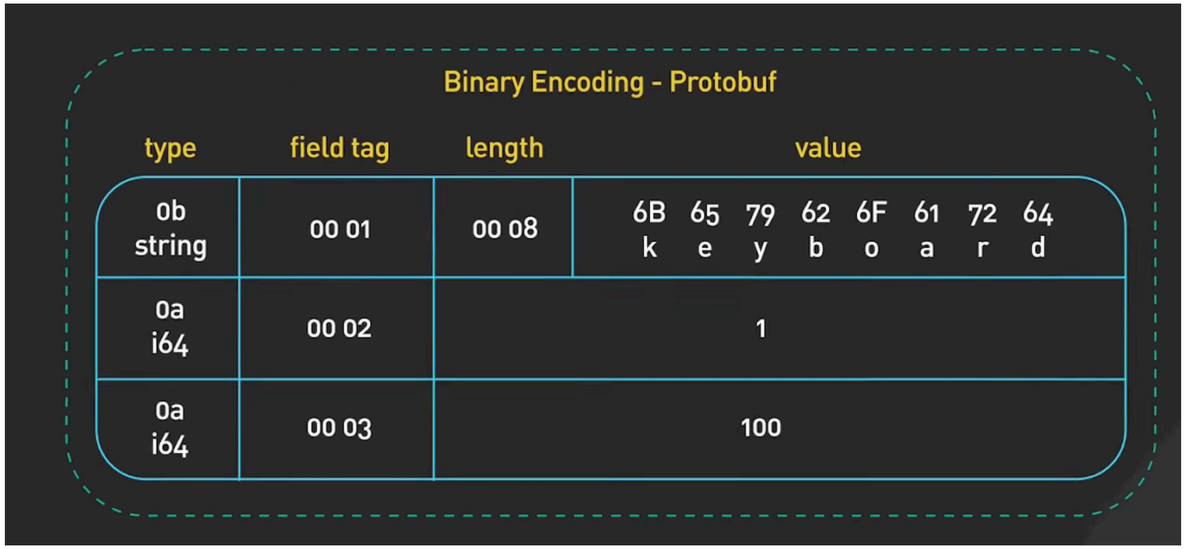
```
service Publisher {
  rpc SignBook (SignRequest) returns (SignReply) {}
}

message SignRequest {
  string name = 1;
}

message SignReply {
  string signature = 1;
}
```



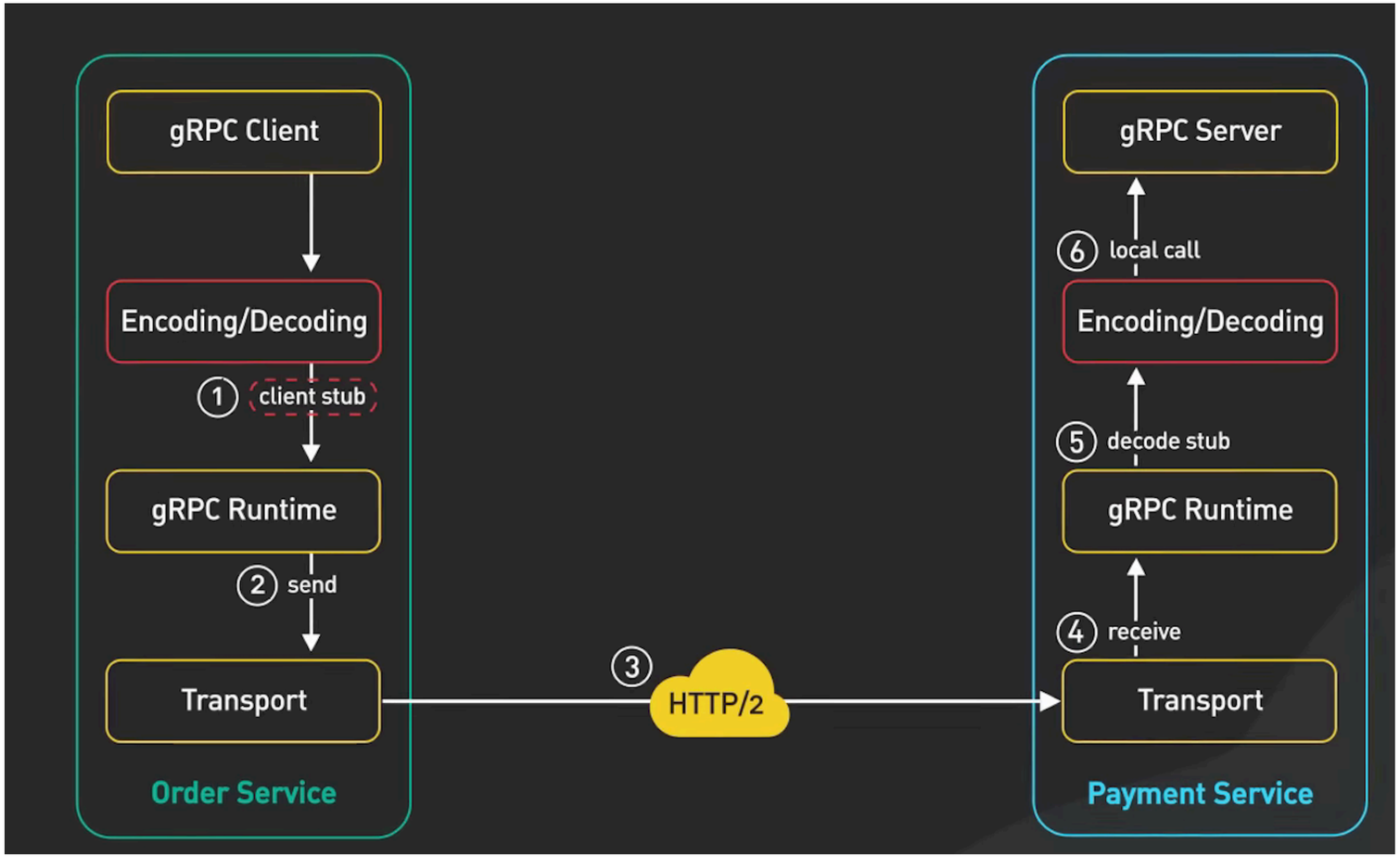
gRPC



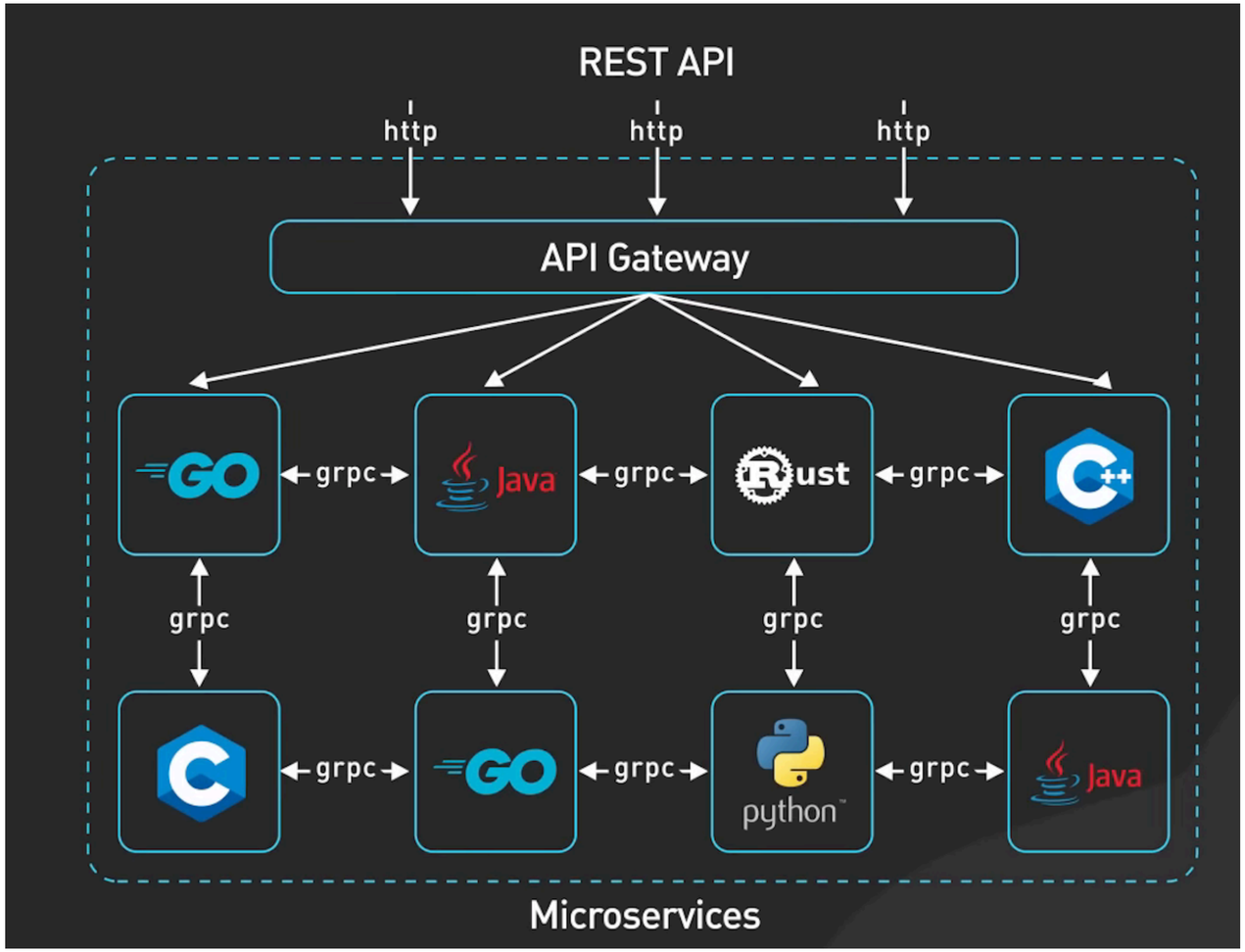
HTTP/2 brings many benefits:

Multiplexing
Stream prioritization
Binary protocol
Server push

gRPC


gRPC Client
Encoding/Decoding
1 client stub
gRPC Runtime
2 send
Transport
Order Service
3
HTTP/2
gRPC Server
6 local call
Encoding/Decoding
5 decode stub
gRPC Runtime
4 receive
Transport
Payment Service

gRPC


REST API
http
http
http
API Gateway
GO
grpc
Java
grpc
Rust
grpc
C++
grpc
grpc
grpc
grpc
C
grpc
GO
grpc
python
grpc
Java
Microservices